Repair Estimator — One-Page Writeup

1. Most Interesting Design/UX Decision

Inline search bar with instant navigation. Rather than a modal overlay or separate search page, the search bar lives permanently below the header — always visible, zero taps to reach. As you type, results filter in real-time via targeted DOM swaps (not a full re-render), preserving input focus. Tapping a result switches categories, collapses all groups except the matching one, scrolls the active tab into view, then scrolls the item card into view. This turns a 7-tab, ~130-item tree into a single-keystroke lookup — critical for field agents who need to find "water heater" or "drywall" fast while walking a property. The summary also lives as a tab in the category bar, giving a direct one-tap review path without tabbing through every category.

2. What's Broken or Fragile

- **Single-file monolith** — all HTML/CSS/JS in one 2000+ line file. No module boundaries, no type checking, hard to test.
- **`innerHTML` full re-render** — most state changes wipe and rebuild the DOM. Focus, scroll position, and transient UI state are lost unless explicitly preserved (mitigated by partial DOM updates for search and totals, but fragile).
- **localStorage for photos** — base64 data URLs consume ~1.37× the binary size; a property with 50+ photos hits the ~5–10 MB quota quickly. No compression beyond canvas resize.
- **CDN dependencies** — Tailwind, TensorFlow.js, XLSX, JSZip all load from CDN. First load (or any offline launch) fails silently if uncached.
- **Chromium-only barcode detection** — `BarcodeDetector` API is unsupported on Safari/Firefox, so serial auto-extraction silently does nothing on iOS.
- **Heavy client-side ML** — TensorFlow.js requires a ~2 MB model download on the first photo capture. Inference runs client-side and can be slow on older mobile devices (2–5 seconds per photo). The damage detection pipeline will be profiled and optimized — or swapped for a lighter model — in a future iteration.

3. Creative Addition

AI-Assisted Damage Detection. When an agent captures a photo of a furnace, roof, or water heater, TensorFlow.js (COCO-SSD MobileNet v2) runs client-side object detection to identify items at >25% confidence. Simultaneously, a canvas-based pixel analysis samples ~8k pixels for discoloration and computes a 0–100 damage score. Results appear as color-coded labels on each photo thumbnail: green ✓, yellow ⚠ (>12%), or red ⚠ (>35%). No server upload needed — all inference runs in the browser.

Why I chose this: In house flipping, hidden structural/equipment damage is the #1 budget-killer. A missed furnace or roof replacement can each swing the estimate by $3,000–$10,000+. Giving acquisition agents a real-time, offline AI assistant that flags potential issues on photos reduces the chance of walking past a major expense during the 20-minute walkthrough that determines the offer price.

4. What I'd Ship Next (Two More Days)

- **IndexedDB photo storage** — migrate base64 photos to IndexedDB blobs to bypass localStorage quota limits.
- **Estimate templates** — save a configured estimate as a template and apply it to new projects for recurring property types (e.g., "Standard Cosmetic Flip").
- **Offline-first sync** — lightweight backend for multi-device access; syncs when connectivity returns.
- **Cost breakdown charts** — canvas-based pie/bar charts in the summary view.

5. AI Tools Role

AI assistants (Claude) were used as a pair programmer throughout: generating initial component structure, implementing the live search system with partial DOM updates, refactoring the header layout (responsive, truncation, PDF button), and debugging CSS/event-handling issues like no-action item disabling and swipe gesture false triggers. All AI-generated code was reviewed and iterated on. The architectural decisions — single-file state management, multi-instance room logic, rendering strategy, and client-side ML inference — were human-driven.